

Turbulence MGD runs: regularised θ , numerics and moment matching

Recap of 23–25 September 2026

Chiara Zelco

25 September 2026 (first version 24 September)

Summary

The goal was to redo the turbulence MGD runs with more particles and to choose the regularisation strength λ of the time-regularised estimator $\Theta_{\text{reg}}(t)$. Looking into how λ acts led to a chain of findings: the block averaging used so far (`n_subsample`) did most of the smoothing, the old dense solver carried a hidden bias, and the statistic basis turned out to be exactly rank-deficient. The solver was replaced, λ can now be chosen *after* a run without re-simulating, and several numerical failure modes were fixed. The most important open problem is scientific: in all turbulence runs, including the July reference batch, **the moments are matched to $\sim 10^{-3}$ for $t < 0.95$ but not near $t = 1$** . The runs launched on 24 September (Section 8) show that the moment error *jumps* near the end, at a point set by the step size relative to the time left, $h/(1 - t)$, and that the samples are then physically wrong (too much small-scale energy, no intermittency). Only schedule exponent 3 with ridge 10^{-2} avoided it, and it still misses the 1% target. On 25 September two new time grids were added (a fixed two-phase grid and an adaptive one), each SDE step was made about a third cheaper (expected, not yet measured), and runs testing the new grids were launched (Section 11). No final λ , time grid or potential set is reported here.

Throughout, `[measured]` marks something computed on data or in a controlled test, `[hypothesis]` marks an interpretation not yet tested directly.

1 Starting point: the July reference batch

51 seeds (300–350) of the full potential set (`L.6`, `L.6_psi`, `L.2_lowpass`, `Scalar_psi_gaussianK`, `Scalar_morlet_gaussianK`, fourth-order `Mod2` scattering, real and imaginary, $Q = 1$), with $M = 256$ (1024-sample windows coarse-grained twice), $J = 8$, $Q = 3$, $\sigma = 3.5$, $n_t = 40\,000$, $n_1 = 5000$ particles, $\lambda = 5 \cdot 10^{-7}$, `n_subsample` = 200, per-step ridge (`regularization`) = 10^{-2} . SDE time: 10.6–10.7 h per seed on H100 `[measured]`.

2 How λ and `n_subsample` act on Θ_{reg}

The regularised problem (the time coupling follows Guth et al.’s dual score matching) is, in the continuum limit,

$$M(t)\Theta - b(t) = \lambda \frac{d}{dt}(G(t)\dot{\Theta} - c(t)),$$

with $M = \mathbb{E}[\nabla\phi\nabla\phi^\top]$ and $G = \mathbb{E}[\phi\phi^\top]$ on the walkers. The grid spacing does not appear: λ alone sets a smoothing length $\ell \sim \sqrt{\lambda G/M}$.

- `n_subsample` is *not* part of Guth’s method. It was introduced to keep the dense $(nr)^2$ solve in memory, and it averages M, G, b, c over blocks of n_s steps. The block averaging makes it a second smoother (a boxcar of width $\Delta = n_s/n_t$). The effective window is $\approx \max(\Delta, 4\ell)$.
- In the July runs, $\lambda = 5 \cdot 10^{-7}$ was essentially *inactive* for $t < 0.95$: Θ_{reg} equals the block average of the per-step θ_t to within 0.15–0.37 block standard errors; λ only acts for $t > 0.95$, where the Cos schedule makes the steps small (the coupling is λ/dt^2) [measured] (4 seeds).
- The per-step θ_t noise is white (lag-1 autocorrelation of differences -0.455 , vs. -0.5 for white noise), so averaging does reduce variance as expected [measured].
- Each block was labelled with its *first* time although it holds the block average: a half-block time shift [measured] (toy with known θ).

3 Solver: from dense float32 to block-Thomas float64

Problems of the old dense solve [measured] (tests with the actual solver code)

- The ridge $0.01 \cdot I$ was added *after* Jacobi rescaling, i.e. $0.01 \text{diag}(A)$ in original units, which contains the coupling λ/dt^2 . For $\ell/\Delta \gtrsim 2$ it shrinks Θ towards zero (toy with a known θ of RMS 0.87: RMS error 0.82 at $\lambda = 5 \cdot 10^{-5}$, $n_s = 10$, vs. 0.004 without this ridge). So smaller `n_subsample` was *not* better at fixed λ .
- It ran in float32; on ill-conditioned systems with large λ the error reached 640% (the block-Thomas solver in float32 failed similarly).

New solver Block-Thomas elimination, float64, on CPU, $O(nr^2)$ memory, no ridge inside the elimination. The system is symmetric positive definite, so no pivoting across blocks is needed. Matches the exact solution to 10^{-14} – 10^{-8} even at condition number 10^9 [measured]. With `n_subsample = 1` it solves Guth’s system on the full grid (no block averaging). On the Gaussian test case θ is recovered correctly (checked on the cluster). Selected with `--reg_solver thomas`; the old path is unchanged.

4 Choosing λ after the run

λ does not enter the SDE, only the final solve. Runs can therefore save the assembled system $(t, M_k, G_k, b_k, c_k; \sim 25 \text{ GB per run})$ and re-solve it offline for any λ (`--save_reg_system, codes/resolve_theta_reg.py`). The offline solve reproduces the in-run one exactly (difference 0.0) [measured]. On the cluster: 136 s per λ for $r = 235$, $n = 39\,716$ nodes [measured].

Selection rule (`turbulence/lamtune_select.ipynb`) Residual $R = \Theta_{\text{reg}}(\lambda) - \Theta_{\text{reg}}(0)$. While λ only removes noise, block means of R shrink like noise and $\mathbb{E}[z^2] \lesssim 1$; when it removes signal, $\mathbb{E}[z^2] \gg 1$. Choose the largest λ with $\mathbb{E}[z^2] \leq 1$ in every time window. On synthetic data with known $\theta(t)$ this threshold landed on the true RMS-error optimum; a threshold of 2 overshoot by half a decade ($1.6\times$ the optimal error) [measured]. The reference is $\lambda = 0$ of the same solver, not θ_t , whose per-step ridge would appear as spurious structure.

5 Numerical issues found on the turbulence runs

1. **GPU memory.** The order-4 scattering gradient holds ~ 6 complex tensors of shape $(B, 8, 8, 9, 256)$ at once: $\sim 6.6 \text{ MB per sample}$ [measured]. Batches of 5000 fit an 80 GB H100; a 16 GB V100 needs ≤ 1000 .

2. **Throughput.** $\approx 1.9 \cdot 10^{-4}$ s per particle per step on H100, consistent over three batches [measured]. Hence $n_1^{\max} \approx 18 \text{ h} / (n_t \cdot 1.9 \cdot 10^{-4} \text{ s})$: ~ 8500 at $n_t = 40\,000$; $n_1 = 10\,000$ needs $\sim 21 \text{ h}$, above the 20 h limit. (Updated 25 September: about a third of each step was duplicated work, now removed; see Section 10.)
3. **Dead potentials.** Two of five seeds ($n_1 = 10^4$, $n_t = 33\,000$) crashed at step 32 583 (i.e. $t \rightarrow 1$) with a “singular” per-step Gram. With the ridge added after rescaling, a finite matrix cannot be singular, so the cause is a potential whose gradient is exactly zero on all walkers (0/0 in the rescaling) [measured] (no NaN before the crash). Fix: such a coefficient is set to 0 for that step only; healthy steps are bit-identical to before. Most likely an emptied region of a `Scalar.*_gaussianK` statistic [hypothesis].
4. **Rank-deficient statistic basis.** With no ridge at all, every offline solve was singular. The scaled M_k has 2–3 eigenvalues at 10^{-16} – 10^{-13} [measured]. Traced to `L_6_psi[20–23]`: at $M = 256$, $J = 8$, $Q = 3$ the wavelet channels 21, 22, 23 are *exact copies* (all the single frequency bin 1), channel 20 is a near-copy (a $1.1 \cdot 10^{-4}$ component at bin 2) [measured]. Removing exact copies loses nothing ($\theta_a \phi + \theta_b \phi = (\theta_a + \theta_b) \phi$); `--deduplicate_filters` drops them for the channel-list potentials only (25→23 channels). The offline solve uses a ridge $10^{-6} \text{diag}(M_k)$ on the data term. The same happens for the Gaussian bank ($M = 128$, $J = 7$).
5. **Float32 per-step solves.** With near-duplicate statistics, float32 is already $5 \cdot 10^{-4}$ off at ridge 10^{-4} and 66% off at 10^{-8} ; float64 stays at $\sim 2 \cdot 10^{-8}$ [measured]. New option `--solve_float64`.
6. **The basis depends on n_1 .** The `Scalar.*_gaussianK` fit keeps a data-dependent number of region statistics: $r = 235$ at $n_1 = 8500$ vs. $r = 291$ at $n_1 = 10^4$ [measured]. θ is therefore not comparable coefficient-by-coefficient across n_1 .

6 Moment matching fails near $t = 1$

Relative error between target ($\bar{\phi}_e$) and walker ($\bar{\phi}_p$) moments, as in `plot_moment_matching`:

$$\frac{2 |\bar{\phi}_e - \bar{\phi}_p|}{|\bar{\phi}_e| + |\bar{\phi}_p|},$$

full potential set, 512-sample windows coarse-grained once [measured]:

run	mean error per t -window		final step, over moments		
	$t < 0.95$	$[0.95, 1]$	mean	median	p90
$n_1 = 10^4$, $n_t = 33\,000$ (3 seeds)	$1\text{--}1.8 \cdot 10^{-3}$	0.28	1.87	2.00	2.00
$n_1 = 8500$, $n_t = 40\,000$ (5 seeds)	$0.9\text{--}1.5 \cdot 10^{-3}$	0.041	0.28	$1.2 \cdot 10^{-3}$	1.40

The July batch shows the same problem on its per-run moment-matching figures (not yet quantified with the comparison script). A final median of 2.0 is the maximum of this metric.

Interpretation (as of 24 September; see Section 8 for what the runs showed)

- Not a particle-count problem: more particles improve the empirical moments; the run with *more* particles but *fewer* steps is the worse one [hypothesis]. (25 September: n_1 is still the one setting that differs between the $n_1 = 2000$ runs, where ridge 10^{-4} worked, and the $n_1 = 8500$ runs, where it failed; open.)

- **Suspect 1: the per-step ridge.** 10^{-2} only partly corrects directions in which G is nearly degenerate, and G becomes more degenerate near $t = 1$ (duplicate pairs 3→6); the targets also move fastest there, so partial corrections fall behind [hypothesis]. (25 September: at full size the *larger* ridge is the one that helps, and only with exponent 3.)
- **Suspect 2: time resolution near $t = 1$.** Consistent with $n_t = 33\,000$ being worse than 40 000 [hypothesis]. (25 September: supported by the runs, in the form of $h/(1 - t)$. The earlier remark that a larger `schedule_exponent` is free was wrong: the time grid is float32 and its steps near $t = 1$ are quantised, see Section 9.)
- **Suspect 3: statistics dying near $t = 1$,** which cannot be corrected at all [hypothesis]. (25 September: the worst moments at $t = 1$ are matched by the ridge- 10^{-2} run, so they are not uncorrectable.)

7 Runs launched on 24 September

All: `--solve_float64`, `--deduplicate_filters`, $n_t = 40\,000$, seed 900, H100.

label	potentials	ridge	exp.	n_1	purpose
<code>diag_reg*_sched*</code> (4 runs)	full	$10^{-2}, 10^{-4}$	2, 3	2000	clean 2×2
<code>long_<set>_sched3_reg1e-4</code> (4)	all 4 sets	10^{-4}	3	8500	production setting
<code>long_full_sched3_reg1e-2</code>	full	10^{-2}	3	8500	full-size 2×2
<code>long_full_sched2_reg1e-4</code>	full	10^{-4}	2	8500	full-size 2×2

(“exp.” is `schedule_exponent`; ridge is the per-step regularization.) The four potential sets are: full; without `L2_lowpass`; without `Scalar_morlet_gaussianK`; without both. The fourth corner of the full-size 2×2 (ridge 10^{-2} , exponent 2) is the existing $n_1 = 8500$ batch (`lamtune_full_n1_8500`, 5 seeds), run with the earlier code: without `--solve_float64` and `--deduplicate_filters`. The long runs save their system for the offline choice of λ . All long runs finished, in 16–17.6 h [measured].

8 Results of the 24 September runs (25 September)

Single seed (900), $n_1 = 8500$, $n_t = 40\,000$ unless stated. Compared with two tools in `turbulence/`:

- `compare_runs.ipynb`: all statistics, all runs overlaid;
- `compare_moment_matching.py`: moment-matching summary.

Short runs ($n_1 = 2000$). Final-step mean relative error: exponent 2: $3.9 \cdot 10^{-2}$ (ridge 10^{-2}), $3.6 \cdot 10^{-2}$ (10^{-4}); exponent 3: $1.2 \cdot 10^{-2}$, $1.0 \cdot 10^{-2}$ [measured]. At this size exponent 3 with ridge 10^{-4} was best.

Full size ($n_1 = 8500$). Moment error at the final step, and the samples at the smallest scale ($\tau = 1$) as ratio generated/data (1 is perfect) [measured]:

run (exp., ridge)	moments, final step			samples at $\tau = 1$, gen./data		
	mean	p90	# > 1	S_2	S_4/S_2^2	S_6/S_2^3
full, 2, 10^{-2} (<code>lamtune_full_n1_8500</code>)	0.27	1.24	24	5.4	0.12	0.015
full, 2, 10^{-4}	0.34	1.54	29	8.2	0.11	0.011
full, 3, 10^{-4}	0.33	1.53	32	8.1	0.12	0.016
full, 3, 10^{-2}	0.032	0.025	1	0.96	0.99	0.96
no <code>L2_lowpass</code> , 3, 10^{-4}	0.36	1.83	32	8.2	0.12	0.012
no <code>Scalar_morlet</code> , 3, 10^{-4}	0.42	1.93	35	16	0.085	0.005

(Full set: 197 of 233 moments above the threshold. The run without both potentials was not loaded in the comparison. The reduced sets have fewer moments, so their moment columns are not directly comparable with the full set.)

- **Only exponent 3 with ridge 10^{-2} gives correct samples:** structure functions and flatness match at every τ . All other runs have up to $10^4\times$ too much power above frequency ≈ 0.2 (visibly noisy time series) and lose most of the small-scale intermittency [measured]. This includes exponent 2 with ridge 10^{-2} , the configuration used so far, so ridge 10^{-4} alone is not the cause.
- Even that run misses 1%: final mean error $3.2 \cdot 10^{-2}$, $1.8 \cdot 10^{-2}$ averaged over $[0.95, 1)$; its median is $6 \cdot 10^{-5}$, so a few moments dominate [measured]. Before $t \approx 0.9$ ridge 10^{-2} is worse than 10^{-4} (mean error per window $1.0\text{--}1.7 \cdot 10^{-3}$ vs. $2.5\text{--}4.5 \cdot 10^{-4}$, exponent 3), and exponent 2 is $1.5\text{--}2\times$ better than 3 there ($1.8\text{--}2.8 \cdot 10^{-4}$) [measured].
- **The error jumps.** Plotted against $1 - t$ on a log axis, the mean moment error does not grow smoothly: with exponent 2 (both ridges) it jumps to ~ 0.3 at $1 - t \approx 7 \cdot 10^{-4}\text{--}10^{-3}$; with exponent 3 and ridge 10^{-4} it follows the ridge- 10^{-2} run until $1 - t \approx 1.8 \cdot 10^{-4}$, then jumps; with ridge 10^{-2} it never jumps and ends at 0.03 [measured].
- **Worst moments at $t = 1$** (full set, ridge 10^{-4} , both exponents; from the `--top` table of the comparison script): moments 52, 53, 54, 84, 85 end at $0.4\text{--}1.8$ on the walkers against targets of $0.9 \cdot 10^{-2}\text{--}8 \cdot 10^{-2}$ (fine-scale energy not removed); moments 37, 38, 47, 63, 68 end $20\text{--}100\times$ below a target that peaks at $t = 1$ [measured]. The run with ridge 10^{-2} matches the tiny moments ($110\text{--}120$, $\sim 10^{-7}$) within 16%, so these are real mismatches, not a near-zero denominator.
- **The spikes of the moment error in the bulk** (e.g. $t \approx 0.28, 0.60, 0.66$, at the same times in every run) are zero crossings of the target moments [measured]: on the ridge- 10^{-2} run, each of the 45 spikes before $t = 0.95$ is carried by one moment (87–100% of the excess), whose target changes sign within ± 50 steps in 42 cases (the other 3 sit next to a crossing of the same moment). The relative error divides by $|\bar{\phi}_e| + |\bar{\phi}_p| \rightarrow 0$ there. They are shared across runs because the target path depends only on the data and the noise Z (same seed and n_1). 34 of the 201 moments change sign at some point.

A better measure of moment matching. The error on each moment’s own scale,

$$\frac{|\bar{\phi}_e - \bar{\phi}_p|}{\max_t |\bar{\phi}_e|},$$

has no spikes: at the spike times it stays at $0.7\text{--}1.6\times$ its local median [measured]. With ~ 200 moments the mean is carried by a few, so the ensemble is summarised by the median, p90 and maximum over moments. Ridge- 10^{-2} run at the final step: median $2.8 \cdot 10^{-6}$, p90 $1.3 \cdot 10^{-3}$, maximum 0.72; 11 of 197 moments above 1% (indices 37–39, 47, 48, 63, 64, 68, 70, 75, 76) [measured]. So this run meets a 1% target on the p90 but not for every moment. Proposed target: p90 $\leq 1\%$ at every t , plus the list of moments above 1% at the end, named from the fitted potentials. (The adaptive grid uses the largest $|\bar{\phi}_e|$ seen *so far*, the only causal choice; at $t \approx 0$ some targets are near zero, so its first steps see a large error and stay at the smallest step.)

θ at the jump. $\|\theta_t\|$ and $\|\theta_t\| h\sigma^2$ (the correction actually applied) rise smoothly by 4–5 decades from $1 - t \approx 1.5 \cdot 10^{-2}$ in every run, including the one that never jumps; at each jump they only step up by less than $2\times$, together with the moment error and dH [measured]. At the jumps, $h/(1 - t)$ from the schedule formula is $\approx 1.9 \cdot 10^{-3}$ (exponent 2) and $\approx 1.3 \cdot 10^{-3}$ (exponent 3, where float32 makes some steps up to $\sim 30\%$ longer) [measured]. Interpretation: the explicit step becomes unstable once $h/(1 - t)$ exceeds $\sim 1.5 \cdot 10^{-3}$, and ridge 10^{-2} damps it

[hypothesis]. (η_t is not saved, so the predictor could not be checked directly; the absolute size of θ is meaningless since the turbulence potentials are not normalised.)

9 The end of the time grid (25 September)

The cut at $1 - t = 5 \cdot 10^{-5}$ loses nothing. The run stops at the first t that rounds to 1.0000. At that point the Cos interpolant still carries noise with weight $\cos(\pi t/2) \approx 7.9 \cdot 10^{-5}$; its power is 1.1% of the data spectrum at the Nyquist frequency and less at all other frequencies [measured]. The finest scales emerge from the noise between $1 - t \approx 10^{-3}$ (noise $4.5\times$ the data at Nyquist) and $2 \cdot 10^{-4}$ ($0.18\times$) [measured], which is where the runs jump.

Float32 quantises the steps near $t = 1$. Largest error of h on the float32 grid vs. the exact one [measured]:

$1 - t$	exponent 2	exponent 3	exponent 4
$[10^{-3}, 5 \cdot 10^{-4})$	5%	11%	16%
$[5 \cdot 10^{-4}, 2 \cdot 10^{-4})$	7%	19%	33%
$[2 \cdot 10^{-4}, 10^{-4})$	10%	32%	49%
$[10^{-4}, 5 \cdot 10^{-5})$	13%	48%	96%

No step is zero before the cut. Exponent 4 was therefore not tried: its extra resolution near $t = 1$ would be largely quantisation. With exponents 2–4 at $n_t = 40\,000$, $h/(1 - t)$ grows towards the end (exponent 3: $\sim 3.5 \cdot 10^{-4}$ at $1 - t = 10^{-2}$, $\sim 2 \cdot 10^{-3}$ at the cut).

New time grids (codes/time_schedules.py, option `--schedule`):

- **power** (default): the old grid, bit-identical, with the same cut [measured].
- **two_phase**: `n_bulk` uniform steps on $[0, t_{\text{switch}}]$, then steps that each remove the same fraction r of the time left, so $h/(1 - t) = r$ is constant down to $1 - t = 5 \cdot 10^{-5}$. r is fitted to the remaining steps, so `--nt` is still the total (e.g. $n_t = 60\,000$, `n_bulk` = 10 000, $t_{\text{switch}} = 0.9$: $r = 1.5 \cdot 10^{-4}$). Float64.
- **adaptive**: each step size is chosen from the previous step’s moment error ($|\bar{\phi}_e - \bar{\phi}_p|$ divided by the largest $|\bar{\phi}_e|$ seen so far, averaged over moments), before the next noise is drawn, so no step is rejected and redrawn (which would bias the SDE). Growth is capped at $\times 1.2$ per step and $h/(1 - t)$ is kept in $[10^{-5}, 10^{-3}]$ by default. `--nt` is the maximum number of steps; the run stops and saves at 90% of the time limit. Float64.

Both new grids run end to end on small local tests; they have not yet been tested at full size.

10 Cost of a step, and the entropy increment (25 September)

Profile (turbulence/launch/profile_step.sh, cProfile on H100, $n_1 = 8500$, 385 steps): 1.57 s per step [measured]. The Gram matrix was computed three times per step (0.26 s each, $\sim 50\%$), the third time an exact repeat of the corrector’s $G(y_k)$ ($\sim 17\%$); the moment velocity $\frac{d}{dt}\mathbb{E}[\phi(I_t)]$ twice ($\sim 33\%$), the second for the entropy increment ($\sim 16\%$) [measured]. Both repeats are now reused. The expected cost is ~ 1.05 s per step, not yet measured.

The entropy increment. $dH_k = -\theta_k \cdot \dot{m}$ used ϕ on the interpolant at t_{k+1} but its velocity $\dot{I}$ at t_k , unchanged since the 11 June upload with no documented reason. The entropy rate $dS/dt = -\theta_t \cdot \dot{m}_t$ has both at the same time, and θ_k belongs to t_{k+1} ; it now uses t_{k+1} for both, which is also what the next step’s predictor needs.

- Exact case ($\phi = x^2$, target $\mathcal{N}(0, 0.2^2)$, $\sigma^2 = 0.4$, $n_1 = 10^5$, scalar code), error of the integrated dH , old $\rightarrow$ new: $+0.093 \rightarrow +0.069$ ($n_t = 100$), $+0.024 \rightarrow +0.017$ (400), $+0.014 \rightarrow +0.013$ (1600) [measured]. The old version was the least accurate of the variants tried. The form $-\theta_k \cdot (m_{k+1} - m_k)/h$, which a concavity argument supports, did a little better ($+0.034$, $+0.008$, $+0.010$) but is not used: at fine steps the variants agree to $\sim 6 \cdot 10^{-3}$ and the remaining $\sim 10^{-2}$ comes from θ itself (Euler–Maruyama bias and estimation noise).
- Bimodal model: the integrated dH moved by 5.0% ($n_t = 200$) and 0.69% ($n_t = 1000$) [measured]. Entropy bounds from runs before and after this change are not directly comparable.
- Samples, θ , moments and Θ_{reg} are bit-identical to the previous code on a small A/B run [measured]; dH does not enter the SDE.

11 Runs launched on 25 September

All: ridge 10^{-4} (the setting that jumped), $n_1 = 8500$, $n_t = 60\,000$ (cap for `adaptive`), seed 900, `--solve_float64`, `--deduplicate_filters`, system saved for the offline λ .

label	potentials	status
<code>schedtest_two_phase_reg1e-4</code>	full	launched
<code>schedtest_adaptive_reg1e-4</code>	full	launched
<code>schedtest_<grid>_<set>_reg1e-4 (6)</code>	the 3 reduced sets, both grids	prepared (<code>schedule_test.sh</code>)

If the jump is gone, it came from the step size and ridge 10^{-4} becomes usable. 60 000 steps at the expected 1.05 s is ~ 17.5 h, close to the 18 h at which a run is stopped; this is also the first measurement of the new step cost.

12 Open questions and next steps

1. Do the new time grids remove the jump at ridge 10^{-4} , and reach 1% (Section 11)? If so, choose between `two_phase` and `adaptive` on accuracy against steps. If not, check the predictor directly (save η_t) or the conditioning.
2. Why did ridge 10^{-4} work at $n_1 = 2000$ and fail at $n_1 = 8500$? n_1 and the batch size are the only settings that differ.
3. Agree on the 1% target (proposed: own-scale p90 at every t , and the named list of moments above 1% at the end), and find out which statistics the 11 failing moments of the ridge- 10^{-2} run are (names on the cluster).
4. λ per potential set from the offline sweep; then choose the minimal set from sample quality (spectrum, structure functions, histograms, KL), which does not depend on λ . The reduced sets at ridge 10^{-4} from 24 September share the jump, so they do not settle this.
5. Larger n_1 : the data give 30 000 series of 512 samples. Within one 20 h job, $\text{steps} \times n_1 \lesssim 5 \cdot 10^8$ at the expected new step cost, so doubling n_1 halves the steps. Beyond that: checkpoint the SDE state and chain jobs. Whether the order-4 gradient dominates the remaining cost was not separated in the profile.
6. Near-duplicate statistics (channel 20 vs. 21, some `Scalar_psi` region pairs) are kept: they carry a little information. Only their combination is well determined, which matters when interpreting individual θ coefficients.

Code changes (for reference)

23–24 September

- `codes/sde_routines.py`: float64 block-Thomas solver; system saving before the solve; dead-potential guard; optional float64 per-step solves.
- New tools:
 - `codes/resolve_theta_reg.py` (offline λ sweep)
 - `codes/find_duplicate_statistics.py`
 - `turbulence/compare_moment_matching.py`
 - `turbulence/lamtune_select.ipynb`
- `codes/filters_bank.py`, `codes/potentials_builder.py`: `deduplicate_filters`.
- `turbulence/launch/`: one tracked script per experiment plus a shared submission file.
- New run-name tags (`_thomas_nsub<N>`, `_f64`, `_dedupfilt`), so runs with different settings never reload each other. (`regularization` and `schedule_exponent` are still not in the name; the launch scripts use distinct labels.)
- Commits 7075cfb, c7bcc39, 0dad74c.

25 September

- `codes/time_schedules.py` (new): `power`, `two_phase`, `adaptive` grids; `--schedule` and its options in `turbulence/run_SDE.py` (the other entry points still build their own power grid). The schedule is not in the run name either.
- `codes/sde_routines.py`: adaptive grid in `forward_regularised`; reuse of $G(y_k)$ and of the moment velocity; dH_k at t_{k+1} (all marked **CHANGED** 2026-09-25).
- `codes/utils_experiment.py`: passes the adaptive controller, saves the grid actually used.
- `turbulence/compare_runs.ipynb` + `turbulence/run_comparison.py` (new): all runs overlaid on time series, marginals, spectrum, structure functions, increment PDFs, wavelet histograms, moment matching, and the θ check.
- `turbulence/compare_moment_matching.py`: `--top` table of the worst moments; the final-error histogram now counts moments (it used a density on log bins, which was misleading).
- `turbulence/diagnostics.py`: `load_all_experiments` overwrote its own input and always returned nothing; fixed.
- `turbulence/launch/`:
 - `PROFILE=true` and `SCHEDULE_ARGS` in the shared submission file;
 - new: `profile_step.sh`, `read_profile.py`, `schedule_test.sh`;
 - `resolve_lamsweep.sh`: `SYSTEM_PATTERN` can be set from the command line.
- Moment matching in `compare_moment_matching.py` and `compare_runs.ipynb` now uses the own-scale error (median, p90, max) and lists the moments above 1% by name; names are only printed when the rebuilt potentials have the run's number of statistics (without the fitted state the rebuild gives 301 instead of 233, which would mislabel every index).
- `codes/check_moments.py`: `spec_plot` ignored `save=`, so no run ever wrote `spectrum.png`; fixed (one figure, spectrum and histogram).